Faire un projet en : Bash / Powershell / ...

Objectif : automatise une / des taches

🌐 **Réseau / Web**

5. 🌍 Vérificateur de site web

- Vérifie la disponibilité (code HTTP, ping)
- Alerte email ou log si le site ne répond plus
- Possibilité d'intégrer une liste de sites à surveiller

Excellent choix ! Un **vérificateur de site web** est un projet à la fois simple, utile et facilement extensible. On peut le commencer petit, puis ajouter des fonctionnalités comme :

- alertes mail ou Telegram,
- logging,
- monitoring de plusieurs sites,
- rapport HTML ou CSV,
- monitoring avec historique, etc.

🚀 Objectif de base

Un script qui :

1. Prend une ou plusieurs URL(s),
2. Vérifie leur disponibilité (code HTTP ou ping),
3. Loggue le résultat,
4. Alerte en cas de problème (optionnel).

🛠️ Versions possibles

🔧 Version Bash (Linux/macOS)

Utilise curl ou wget.

```
#!/bin/bash


# Liste des sites à surveiller
sites=("https://google.com" "https://example.com" "https://monsite.inexistant")


# Dossier de logs
logfile="site_check.log"
```

```
date_now=$(date '+%Y-%m-%d %H:%M:%S')

# Boucle sur les sites
for site in "${sites[@]}"
do
  http_code=$(curl -s -o /dev/null -w "%{http_code}" "$site")

  if [[ "$http_code" -eq 200 ]]; then
    echo "[$date_now] ✅ $site est en ligne (200 OK)" >> "$logfile"
  else
    echo "[$date_now] ❌ $site ne répond pas correctement (Code: $http_code)" >> "$logfile"
  fi
done
```

✅ **À faire tourner via cron** toutes les X minutes/heures.

🔧 **Version PowerShell (Windows)**

```
# Liste des sites
$sites = @("https://google.com", "https://example.com", "https://monsite.inexistant")
$logFile = "site_check.log"
$dateNow = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

foreach ($site in $sites) {
  try {
    $response = Invoke-WebRequest -Uri $site -UseBasicParsing -TimeoutSec 10
    $status = $response.StatusCode
    if ($status -eq 200) {
      Add-Content -Path $logFile -Value "[$dateNow] ✅ $site est en ligne (200 OK)"
    } else {
      Add-Content -Path $logFile -Value "[$dateNow] ⚠️ $site a retourné le code $status"
    }
  }
```

```
  catch {
    Add-Content -Path $logFile -Value "[$dateNow] ❌ Échec de connexion à $site : $_"
  }
}
```

✅ À planifier avec le **Task Scheduler** pour exécution automatique.

🔔 Extensions possibles

1. **Notification par mail**
   - Bash : via mailx ou sendmail
   - PowerShell : Send-MailMessage
2. **Envoi Telegram**
   - Utiliser l'API Telegram Bot pour envoyer une alerte en cas d'erreur.
3. **Interface web simple**
   - Générer un fichier HTML avec les statuts de chaque site.
4. **Export CSV pour analyse**

    - Enregistrer chaque vérif dans un .csv pour analyse ou visualisation (ex: Excel, Grafana).
5. **Historique et graphiques**
    - Stocker les temps de réponse pour faire un suivi des performances.

Souhaites-tu que je t'aide à :

- Personnaliser le script pour ton cas ?
- Ajouter des alertes (mail, Telegram) ?
- Générer une version avec interface HTML ou export CSV ?

Dis-moi ce que tu veux ajouter/améliorer !

Actuellement :

```
Desktop    Downloads        Music     Public           site_check.log  Templates
Documents  glpi-10.0.6.tgz  Pictures  site_checker.sh  snap            Videos
root@Ubuntu22:/home/utec# chmod +rw site_check.log
root@Ubuntu22:/home/utec# chmod +x site_checker.sh
root@Ubuntu22:/home/utec# ./site_checker.sh
./site_checker.sh: line 7: site_check.log: command not found
./site_checker.sh: line 15: syntax error near unexpected token `)'
./site_checker.sh: line 15: `echo "[$date_now] $site est en ligne (200 OK)" >> "$site")'
root@Ubuntu22:/home/utec# cat logs/site_check.log
cat: logs/site_check.log: No such file or directory
root@Ubuntu22:/home/utec# nano site_checker.sh
root@Ubuntu22:/home/utec# chmod +x site_checker.sh
root@Ubuntu22:/home/utec# ./site_checker.sh
./site_checker.sh: line 7: site_check.log: command not found
./site_checker.sh: line 15: syntax error near unexpected token `)'
./site_checker.sh: line 15: `echo "[$date_now] $site est en ligne (200 OK)" >> "$logfile")'
root@Ubuntu22:/home/utec#
```

Site_checker.sh

Non fonctionnel

```
# verificateur de site web


# config
sites=("https://google.com" "https://data.gouv.fr")

logfile= "site_check.log"
date_now=$(date '+%Y-%m-%d %H:%M:%S')

#TEST
for site in "${sites[@]}"; do
https_code=$(curl -s -o /dev/null -w "%{http_code}" "$site")

if [[ "$http_code" -eq 200 ]]; then
echo "[$date_now] $site est en ligne (200 OK)" >> "$logfile")
else:
echo "[$date_now] $site ne repond pas correctement (Code: $http_code)" >> "$logfile"
fi
done
```

28/12/2025 :

```bash
#!/bin/bash
# Verificateur de site web

set -euo pipefail

# Configuration des sites à vérifier
sites=("https://www.example.com" "https://httpbin.org/status/200" "https://www.wikip
logfile="site_check_output.log"

# Création du fichier s'il n'existe pas, mais sans l'effacer
touch "$logfile" || { echo "Impossible d'écrire dans $logfile"; exit 1; }

# Vérification des sites
for site in "${sites[@]}"; do
  date_now=$(date '+%Y-%m-%d %H:%M:%S')

  # Utilisation de curl pour vérifier le code HTTP
  http_code="$(curl -s -o /dev/null -w '%{http_code}' "$site" 2>/dev/null || true)"
  http_code="${http_code:-000}"

  if [ "$http_code" -eq 200 ]; then
    echo "[$date_now] $site est en ligne (200 OK)" >> "$logfile"
  else
    echo "[$date_now] $site ne répond pas correctement (Code: $http_code)" >> "$logf
  fi
done

echo "Vérification terminée. Résultats dans $logfile"
```

La fonction ‘curl’ permet de vérifier l’état du site :

Voici les solutions possibles :

🔍 Comment `curl` détermine le type d’erreur

1) Si le serveur répond, `curl` renvoie un code HTTP réel

Exemples :

Situation	Code renvoyé
Page trouvée	200
Page inexistante	404
Erreur serveur	500
Service indisponible	503